Intent-Based Networking Engine for an ISP

Networks and Systems Management — GRS Project

Miguel Filipe Coelho Mateus — up202206944

Context

An Intent-Based Networking (IBN) Engine

A Python system that reads a YAML file describing the desired network state and automatically provisions the entire infrastructure, routing protocols, DNS, load balancing, and SDN traffic engineering, without any manual configuration.

Key Principles

- Declarative, not imperative
- Engine computes the delta
- Applies only what changed
- Idempotent and repeatable

Motivation

Manual configuration is error-prone

Engineers configure routers, DNS, and routing protocols by hand. Only one typo can bring down a network segment. There is no safety net.

No single source of truth

Network state is scattered across scripts, wikis, and people's memory. There is no declarative representation of what the network should look like.

Slow reaction to network events

When a link degrades, human intervention or slow OSPF convergence is the only response. Traffic engineering is reactive, not proactive.

Key Benefits

- **Efficiency & Speed:** Automated provisioning eliminates manual CLI setup time. Closed-loop SDN reactions happen in sub-second timeframes, bypassing slow routing convergence.
- **Reliability & Safety:** Idempotent operations apply only what changed, ensuring no configuration drift. This eliminates typo-induced outages.
- **Single Source of Truth:** The YAML file acts as the ultimate, easily auditable state of the entire network infrastructure.
- **Reproducibility:** The combination of Python and Docker Compose enables rapid lab recreation, testing, and deployment.

Technologies and services

Python 3

IBN engine, Jinja2 templates, Netmiko SSH automation, Ryu REST integration, Paramiko

Docker Compose

Container orchestration — routers, servers, DNS, clients, SDN controller

Quagga / FRR

OSPF (intra-AS link-state) and BGP (inter-AS path-vector) routing daemons

BIND9 DNS

Authoritative DNS; zone files auto-generated by IBN engine from YAML intent

Nginx

Web server (4 replicas) and HTTP load balancer; upstream config auto-generated

Nagios Core

Active monitoring: latency, BGP sessions, availability; drives closed-loop logic

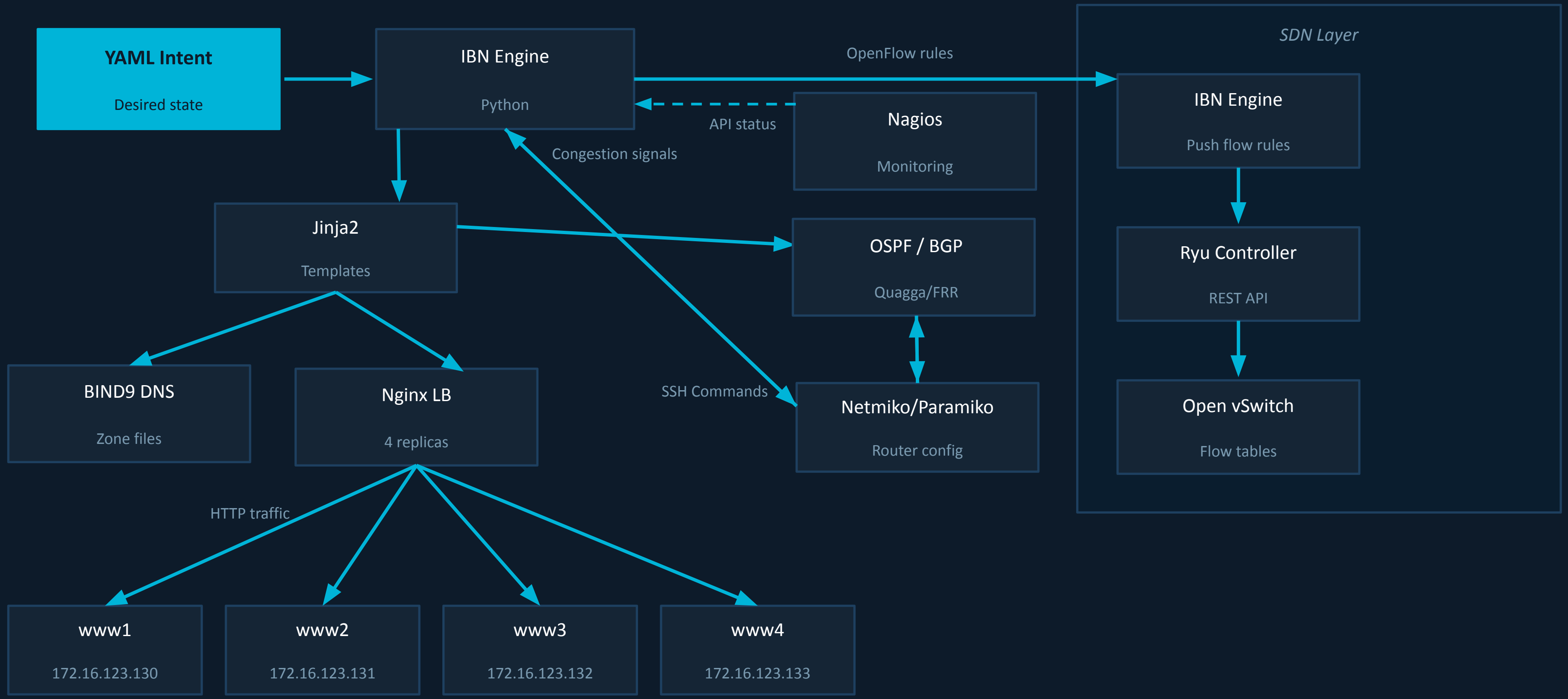
Ryu + OpenFlow

SDN controller managing OVS flow tables; enables sub-second traffic rerouting

Fast API

Dashboard

Architecture Overview



Conclusion

What was achieved

- Successfully moved from manual configuration to a single, declarative source of truth (YAML).
- Achieved automated, idempotent provisioning of routing protocols (Quagga) and core services (DNS, Nginx).
- Demonstrated fast, closed-loop traffic engineering using Nagios and Ryu SDN.

Future Work

- **Alternative SDN Controllers:** The API abstraction allows swapping out the Ryu controller for larger, enterprise-grade controllers like ONOS or OpenDaylight.
- **Docker API & Kubernetes:** The engine's core logic could be extended to translate YAML intent into Kubernetes Custom Resource Definitions (CRDs) or integrate the Docker API to spawn containers dynamically.
- **Enhanced Intent Validation:** Add role-based access control for intent modifications, strict schema validation, and an intuitive web-based UI editor.

Demo and Questions